

DS5899: Special Topics in Data Science

Leveraging NLP in Asset Management

Week 4: Memory

Instructor: Che Guan, Ph.D.

DeepSeek R1 & V3.2 Architecture Deep Dive

Part 1 of 16 in the DeepSeek Implementation Series - The Efficiency Revolution

THE EFFICIENCY REVOLUTION



TRADITIONAL SCALING

Brute-force parameters
Dense $O(L^2)$ Attention
Full KV Cache ($O(L^2)$)
Dense MLP

- \$50M-\$100M+



DEEPSEEK APPROACH

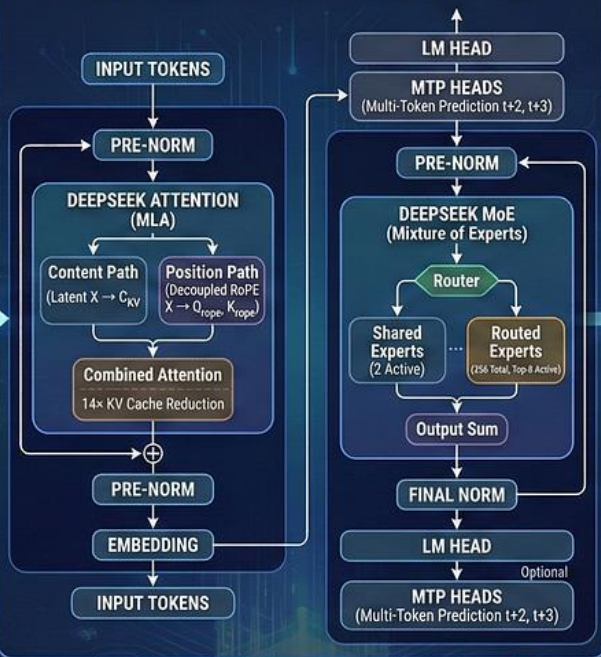
Efficient Architecture
Sparse $O(L)$ Attention
Compressed Latent Cache (14x Reduction)
256-Expert MoE (8 Active)

- \$5.5M for 671B

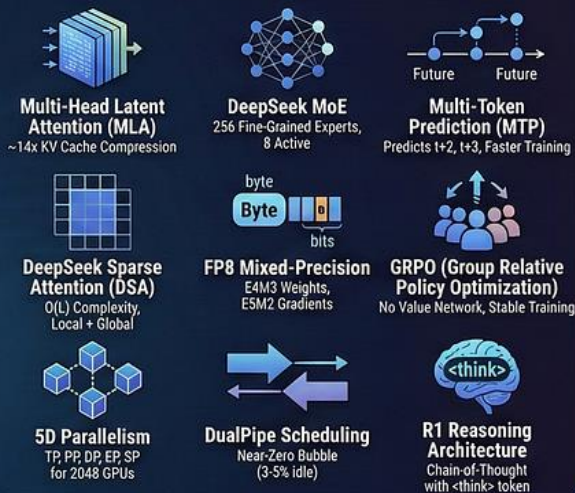
MODEL FAMILY EVOLUTION



DEEPSEEK TRANSFORMER BLOCK



9 BREAKTHROUGH COMPONENTS



THREE-BACKEND PHILOSOPHY



Clone the Repository & Start Training: github.com/DevJadhav/deepseek-from-scratch

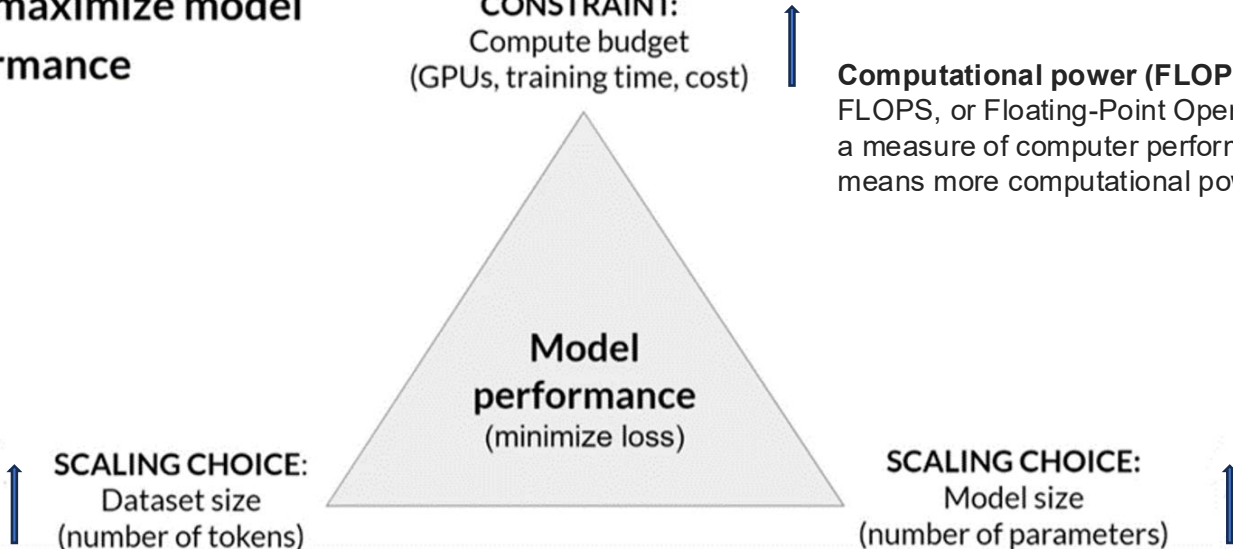


Scaling Choices for Pre-training

Goal: maximize model performance

CONSTRAINT:
Compute budget
(GPUs, training time, cost)

Computational power (FLOPS):
FLOPS, or Floating-Point Operations Per Second, is a measure of computer performance. More FLOPS means more computational power.



Model performance
(minimize loss)

SCALING CHOICE:
Dataset size
(number of tokens)

SCALING CHOICE:
Model size
(number of parameters)

Scaling Laws for Neural Language Models

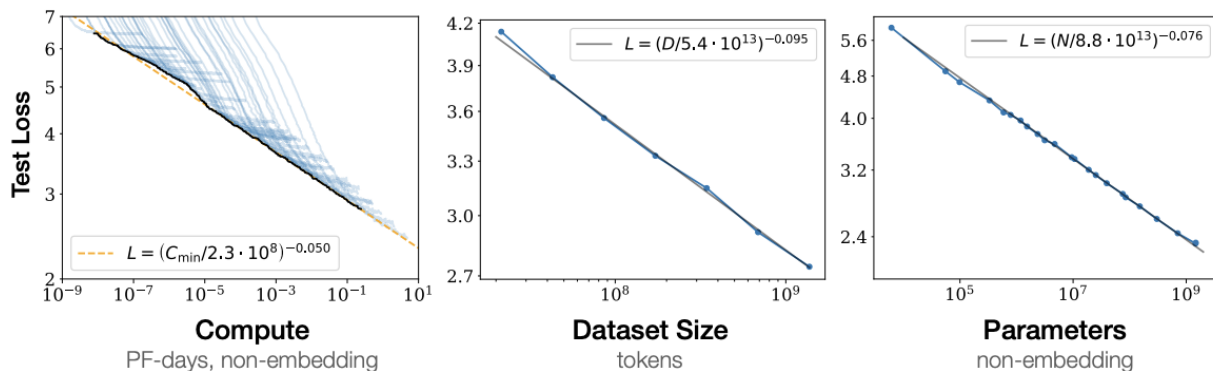
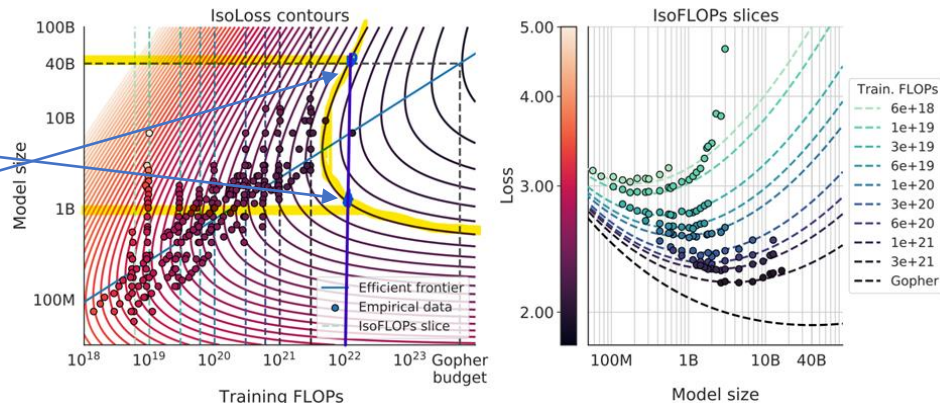


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

- If the compute budget grows tenfold, Kaplan's law suggests that optimal performance is achieved by increasing the model size by 5.5 times and the number of training tokens by 1.8 times.

Is It Possible for Smaller Language Models to Outperform Larger Ones?

- Very large models may be over-parameterized and under-trained. The Chinchilla paper states training data has to increase in the same proportion as the model size to be compute optimal.
- For a compute budget of 10^{22} FLOPS, the performance of a LLM with 1B parameters trained on more data is equivalent to that of a model with 40B parameters. This suggests that increasing the amount of training data can compensate for a smaller model size.



Parameters vs FLOPs: Scaling Laws for Optimal **Sparsity** for Mixture-of-Experts Language Models

- Increasing model **Sparsity** while expanding total parameters reduces pretraining loss under a fixed compute budget.
- Sparsity level: the fraction of inactive parameters, impacts model's performance during pretraining and downstream few-shot evaluation.

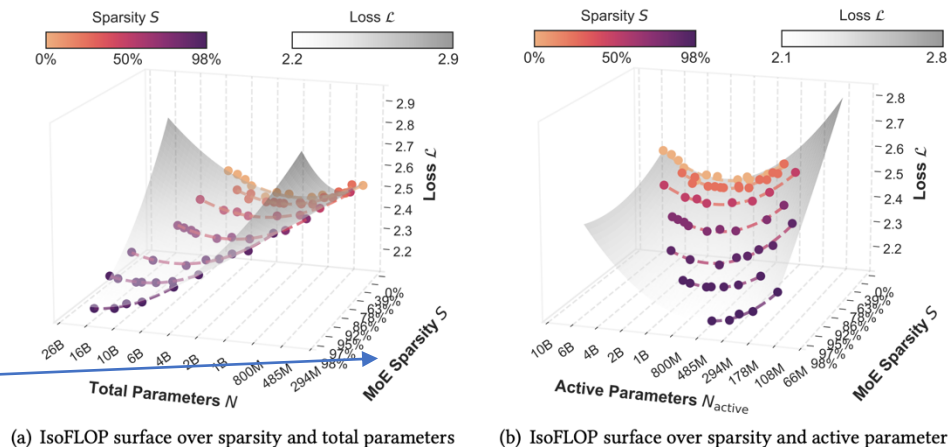


Figure 1: **IsoFLOP surface over observed pretraining loss $\mathcal{L}$, model size (in terms of total N and active parameters N_a), and sparsity S .** We fit a polynomial function mapping N (or N_a), S , and their interaction to $\mathcal{L}$, using empirical data. For both fits the MSE loss for predicting loss on a held out set is 0.0001. These results indicate that for a fixed compute budget, increasing model sparsity leads to a reduction in pretraining loss. When considering optimal model size, we observe opposite trends for total parameters (N) (Figure a) versus active parameters (N_a) (Figure b). (See Figure 8 in Appendix D.1 for results with different total compute budgets C .)

What does it Mean for Pre-training?

Goal: maximize model performance

CONSTRAINT:
Compute budget
(GPUs, training time, cost)



Model performance
(minimize loss)



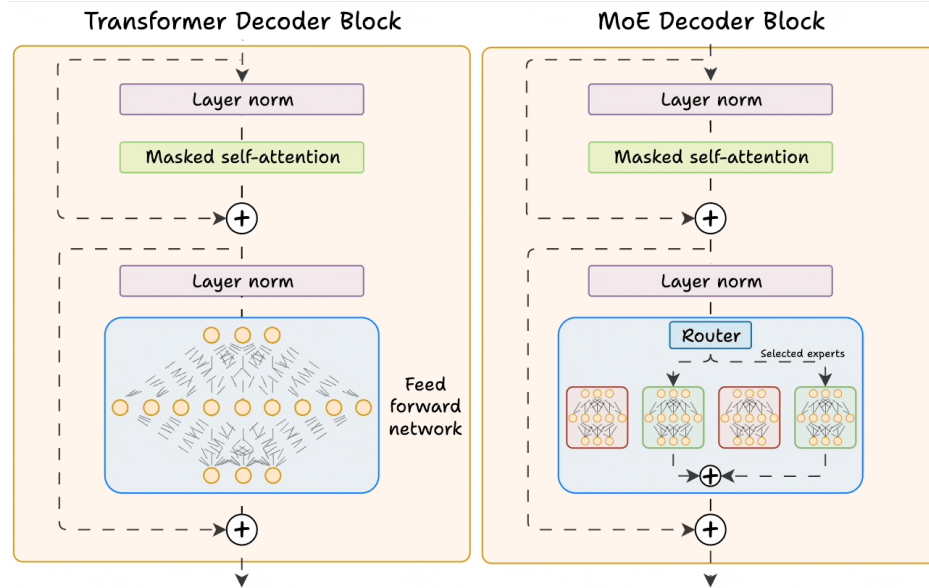
SCALING CHOICE:
Dataset size
(number of tokens)



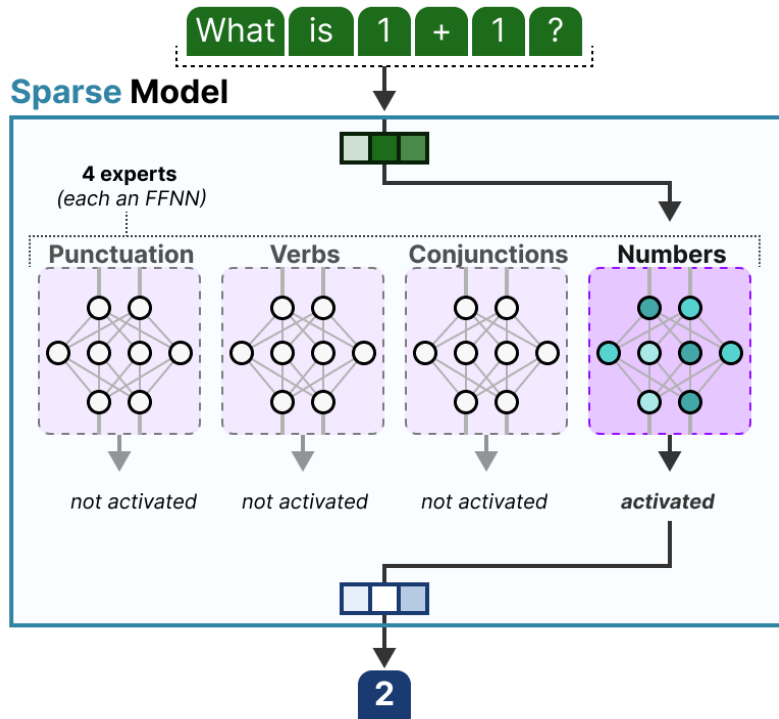
SCALING CHOICE:
Model size
(number of parameters)

MoE in LLMs

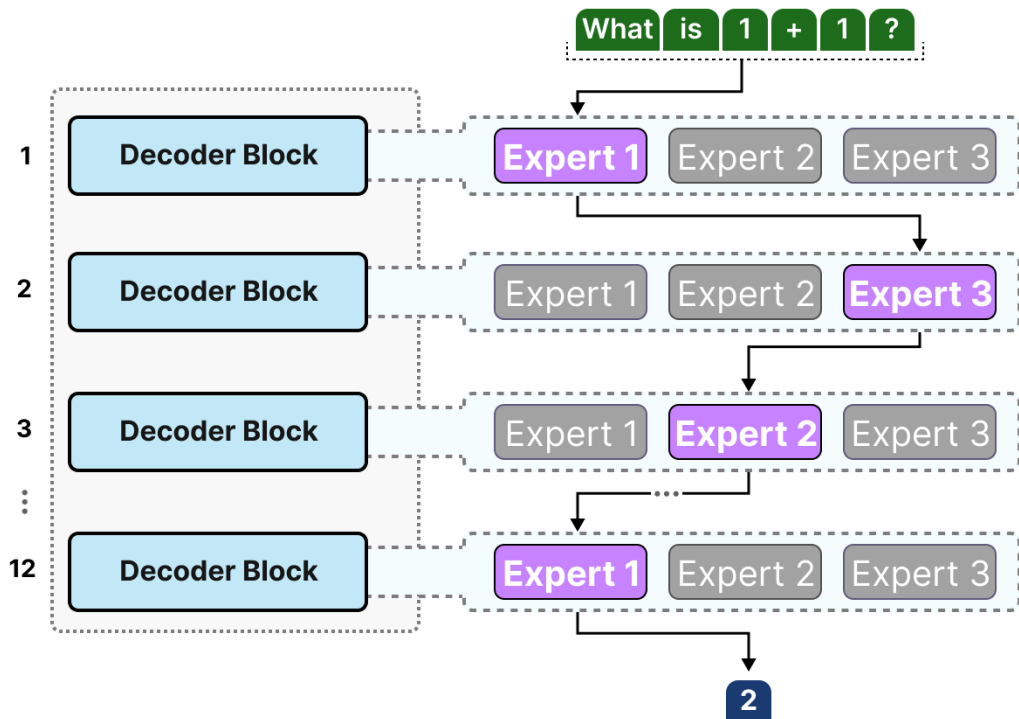
- Mixture of Experts (MoE) is a popular architecture that uses different "experts" to improve Transformer models.
 - Transformer uses a feed-forward network.
 - MoE uses experts, which are feed-forward networks but smaller compared to that in Transformer.
- During inference, a subset of experts are selected. This makes inference faster in MoE.



Visualize Experts

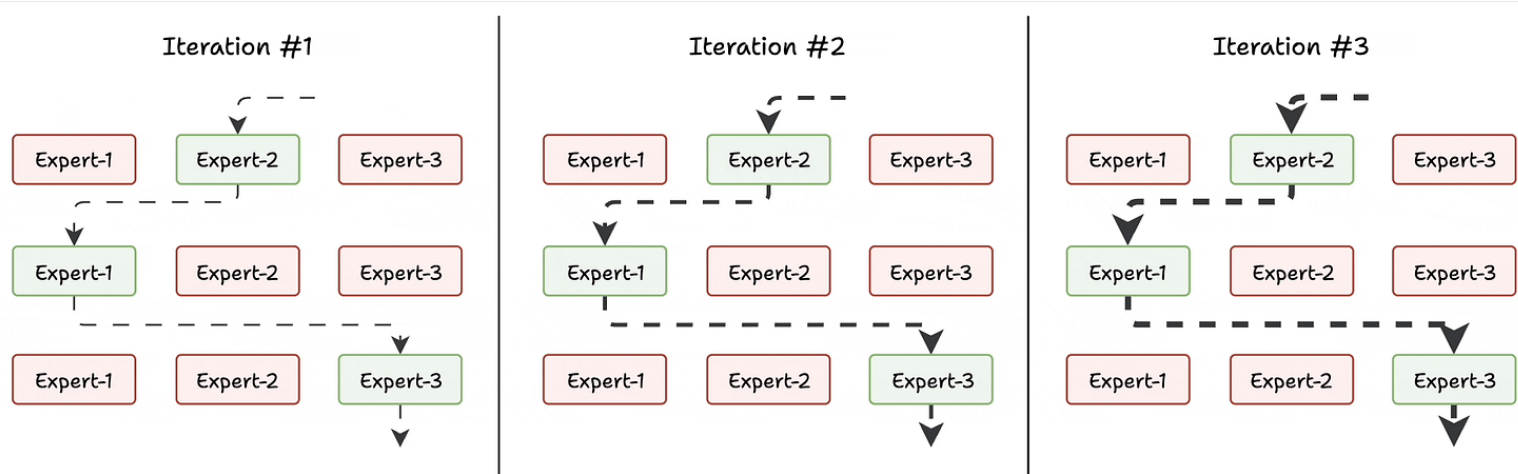


- A given text will pass through multiple experts before the text is generated.



Challenge 1: Repetitive Expert Selection

- The same expert can get selected repeatedly, so we solve it via:
 - Add noise to router FFN output so that other experts can get higher logits.
 - Set all but top **K** logits to **-infinity**. After SoftMax, these scores become 0.



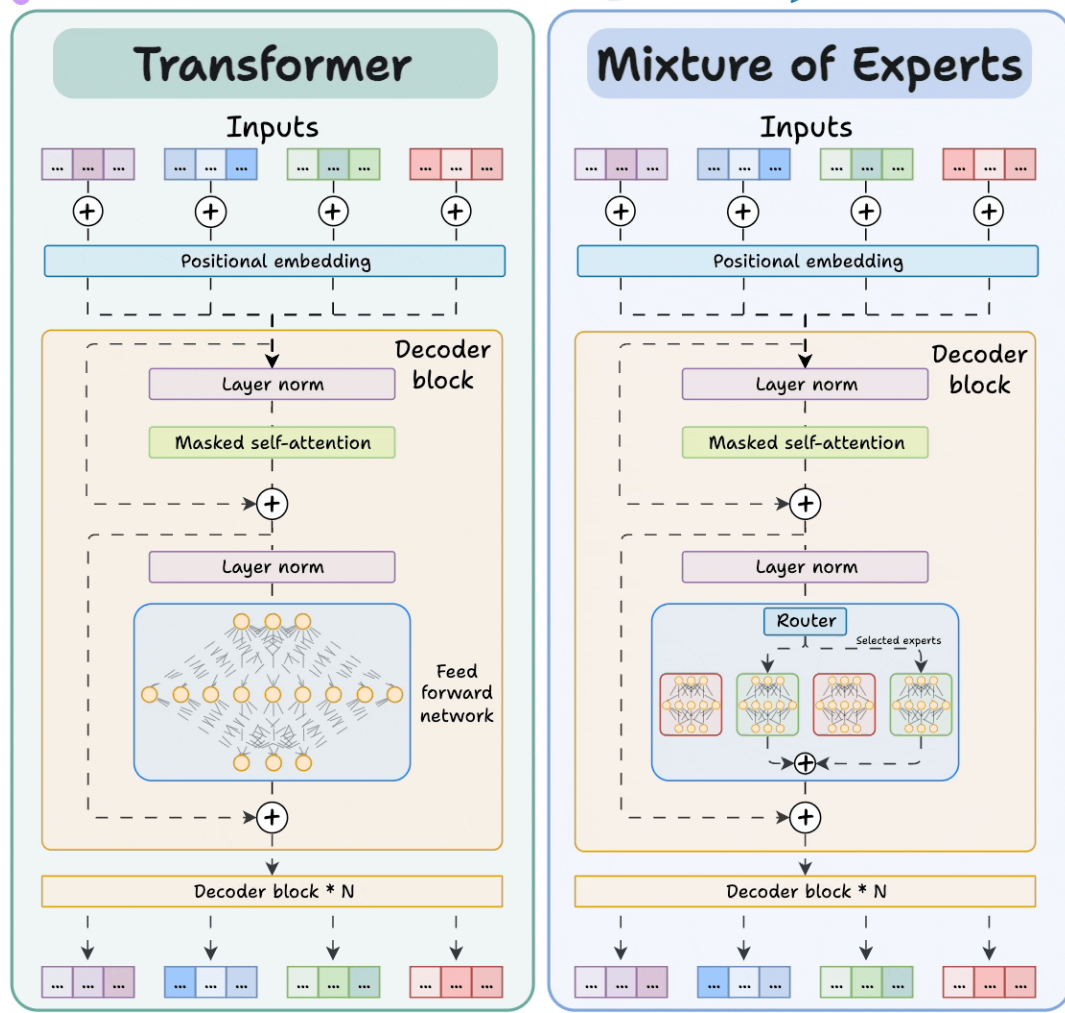
Same experts are getting selected repeatedly as they are getting more training

Challenge 2:

Token exposure disparity

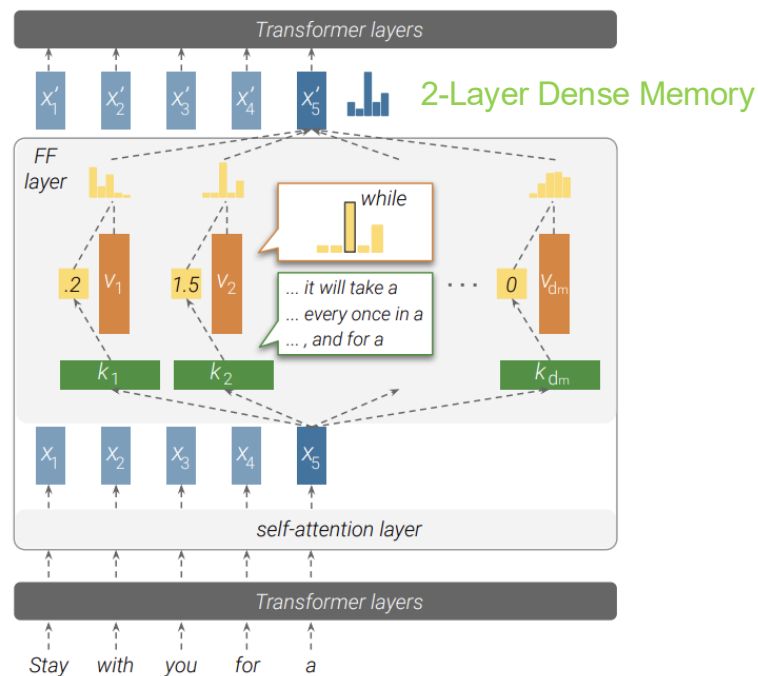
- Some experts may get exposed to more tokens than others—leading to under-trained experts.
 - We prevent this by limiting the number of tokens an expert can process.
 - If an expert reaches the limit, the input token is passed to the next best expert instead.

Transformer vs. Mixture of Experts

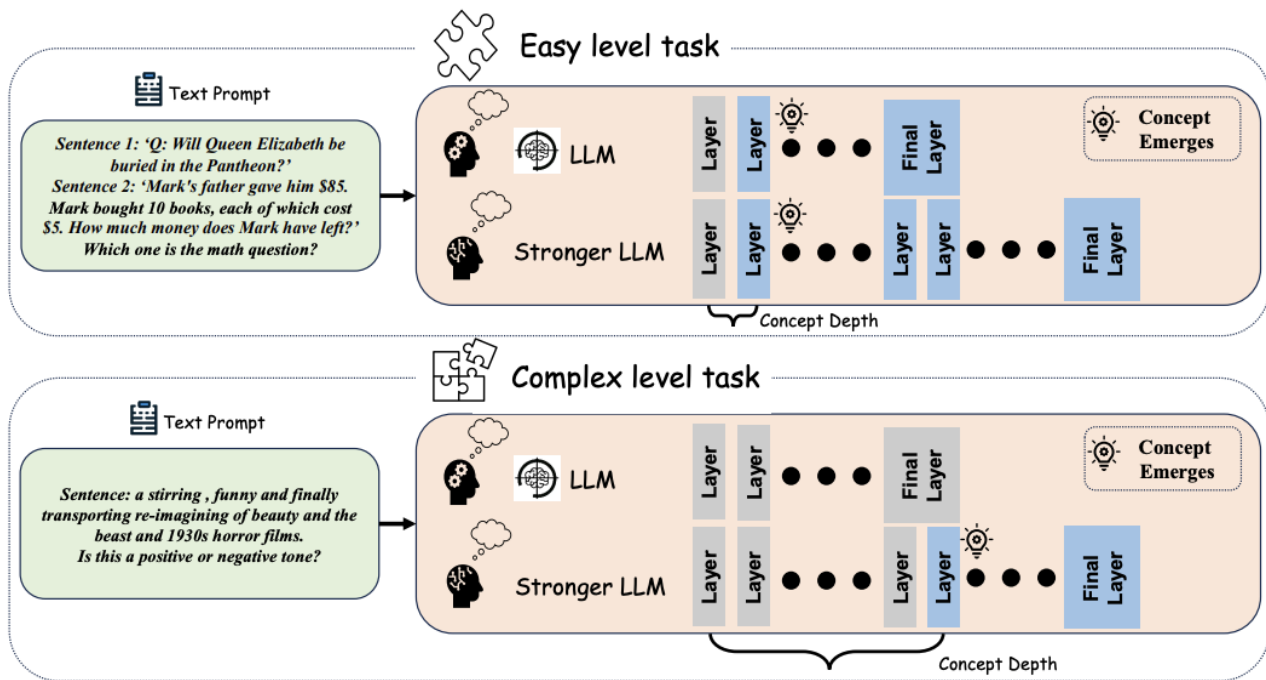


Transformer Feed-Forward Layers Are Key-Value Memories

- An illustration of how a feed-forward layer emulates a key-value memory.
 - Input vectors (e.g., x_5) are multiplied by keys to generate memory coefficients (e.g., a memory coefficient of 0.2 for v_1).
 - These memory coefficients then weigh distributions over the output vocabulary, which is stored in the values.
 - The feed-forward layer's output is thus the weighted sum of its values.



How Large Language Models Acquire Knowledge and Concepts at Different Layers?



LLM need to understand deep layers!

Figure 2: The LLMs are trying to understand easy tasks and complex tasks. The more complex the task, the deeper layers an LLM needs to understand. The stronger the LLM is, the more complex the task level it can learn.

Large Memory Layers w/ Product Keys

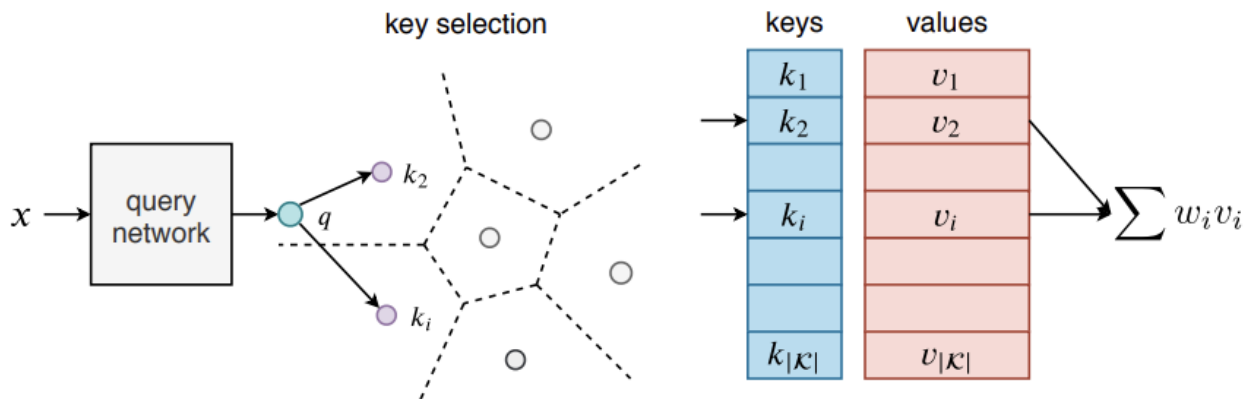


Figure 1: **Overview of a key-value memory layer:** The input x is processed through a query network that produces a query vector q , which is compared to all the keys. The output is the sparse weighted sum over the memories associated with the selected keys. For a large number of keys $|\mathcal{K}|$, the key selection procedure becomes too expensive in practice. Our product key method is exact and makes this search process very fast.

Large Memory Layers with Product Keys, Continued

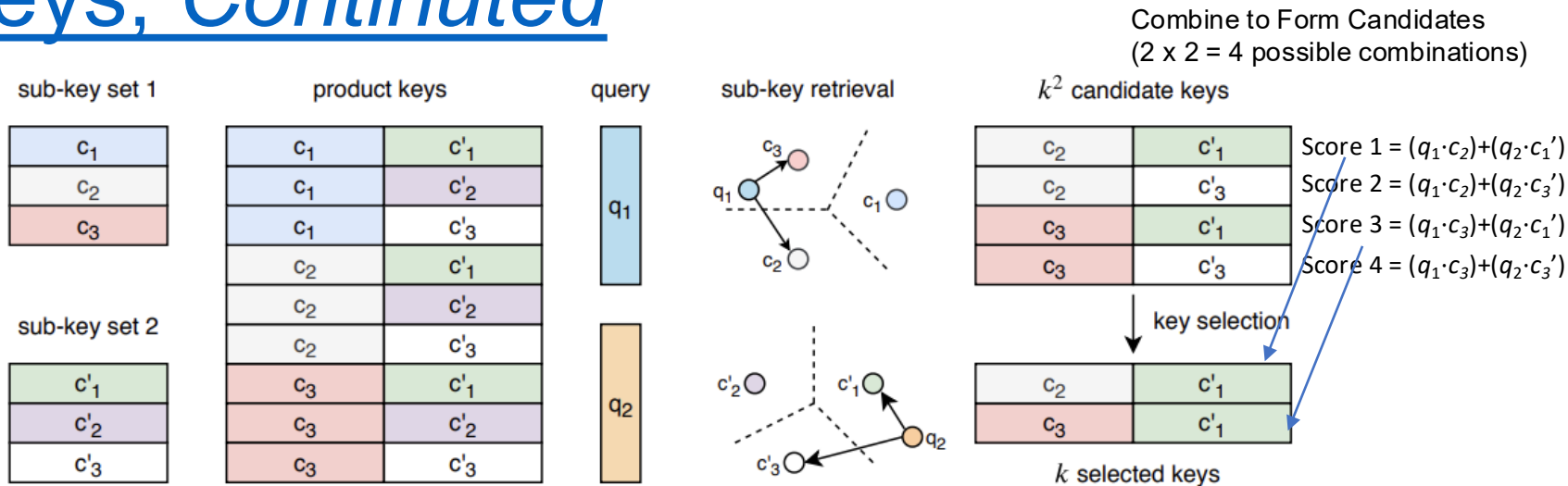
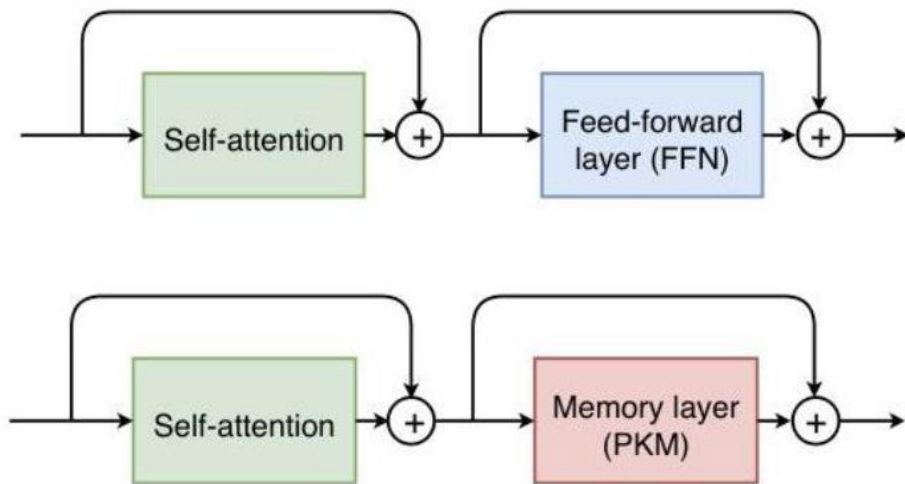


Figure 2: Illustration of the product keys. We define two discrete subsets of keys (sub-key set 1 and sub-key set 2). They induce a much larger set of keys, which are never made explicit (product keys). Given a query, we split it into two sub-queries (q_1 and q_2). Selecting the k closest keys ($k = 2$ in the figure) in each subset implicitly selects $k \times k$ keys. The k keys maximizing the inner product with the query are guaranteed to belong to this subset, on which the search can be done efficiently.

Integration of PKM (Product Key Memory) into Transformer Architecture



- Replacing the traditional FFN layer with PKM (a lookup table) while maintaining residual connections.

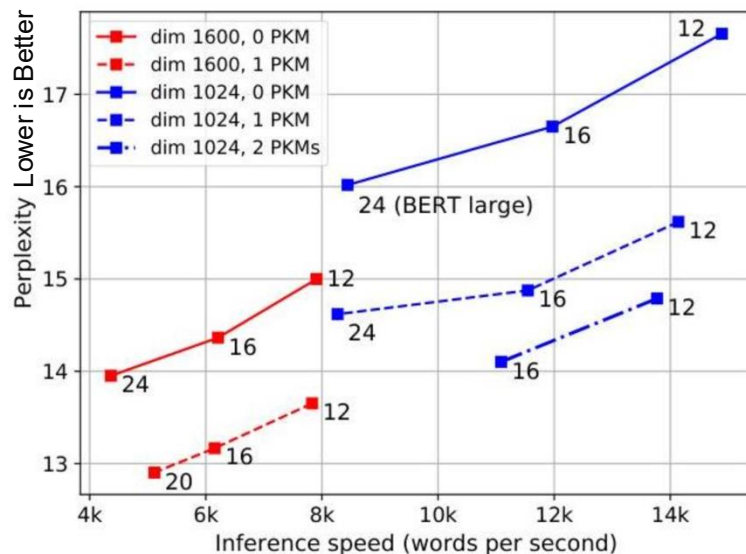


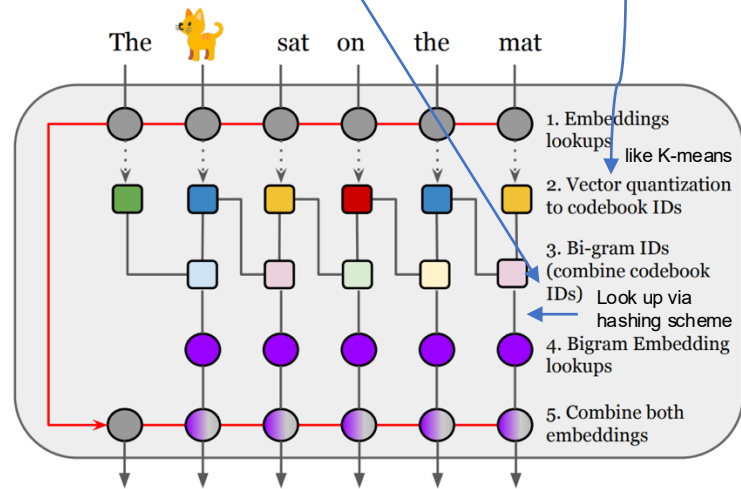
Figure 4: Perplexity vs. inference speed comparison. Models with PKM (blue) achieve better perplexity at significantly higher inference speeds compared to baseline models (red). Numbers indicate layer count.

N-Grammer: Augmenting Transformers with Latent N-Grams

1. Accepts a sequence of unigram embeddings as input.
2. Outputs a parallel sequence of N-gram augmented embeddings.
3. Clusters input embeddings into a discrete latent representation using Product Quantization (PQ).
4. Computes n-gram (bi-gram) IDs from the clustered representation.
5. Retrieves trainable embeddings for each n-gram ID from an embedding table.
6. Combines these embeddings with the input embeddings to generate the output.

- A unigram vocabulary size of 32K (tokens) resulting in a bigram vocabulary size of 1B.
- The Quantization reduces the vector size to 4K, resulting in a bigram size of 16M (rows).

IDs	Codebook
	[0.1, 0.2, ..., 4.0]
	[1.0, 0.3, ..., 1.2]
	[1.1, 3.2, ..., 1.1]
	[5.5, 0.6, ..., 7.0]



Scaling Embedding Layers in Language Models

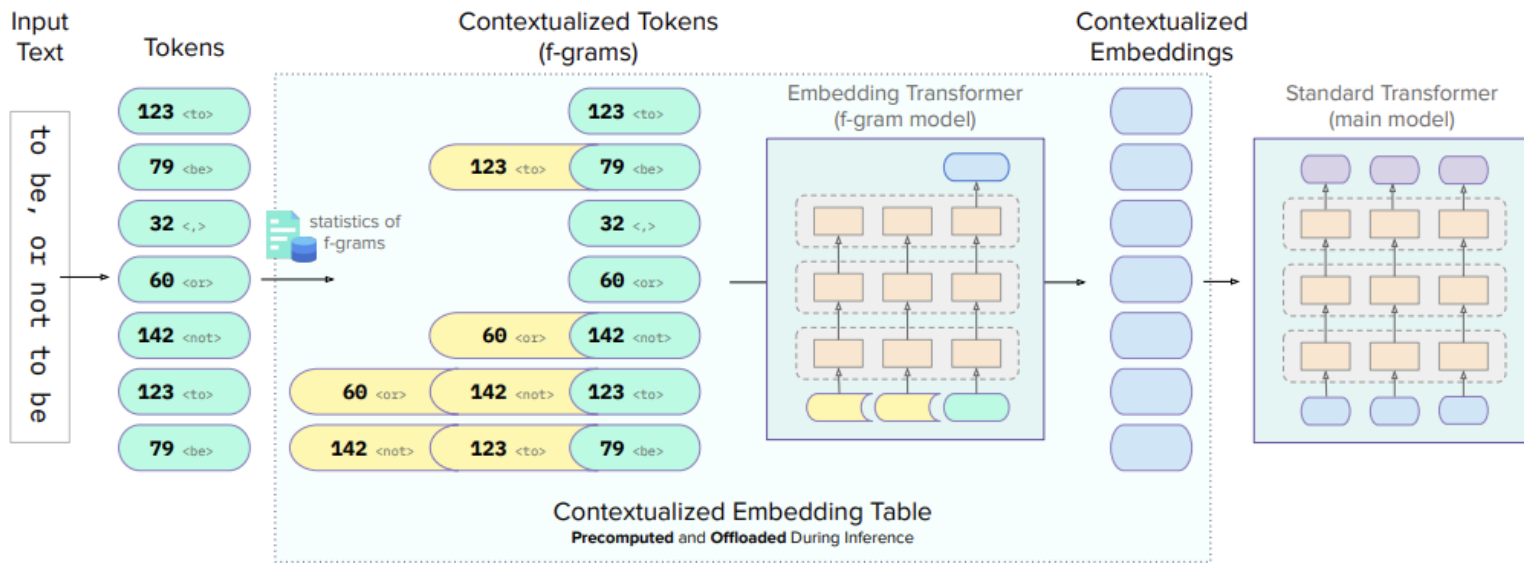


Figure 1: Illustration of Scone with a maximum n -gram length of 3. The f -grams are a set of frequent n -grams selected using the method described in Section 3.1.

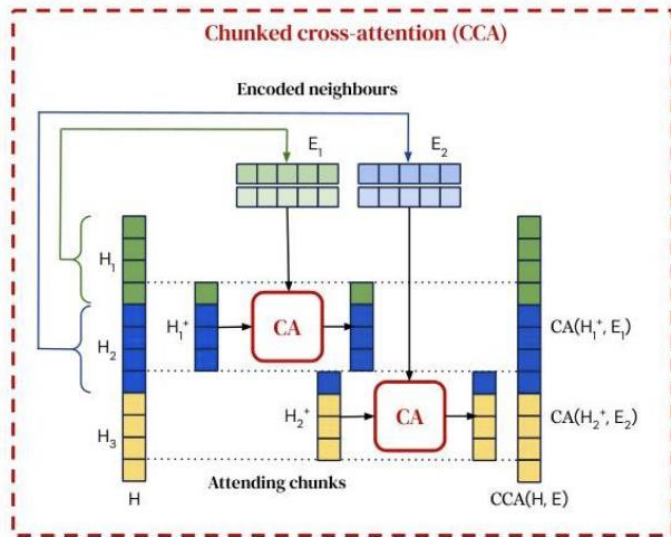
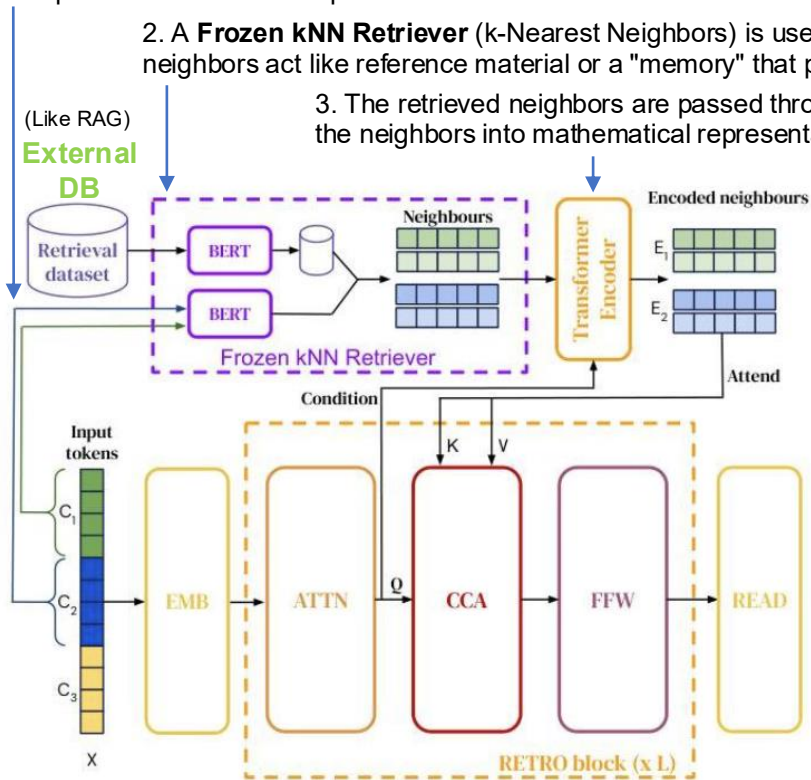
Improving Language Models by Retrieving from Trillions of Tokens

1. A sequence of 12 tokens is split into 3 chunks of 4 tokens each

2. A **Frozen kNN Retriever** (k-Nearest Neighbors) is used to find similar pieces of text, which are called "Neighbors." These neighbors act like reference material or a "memory" that provides context relevant to what the model is currently writing.

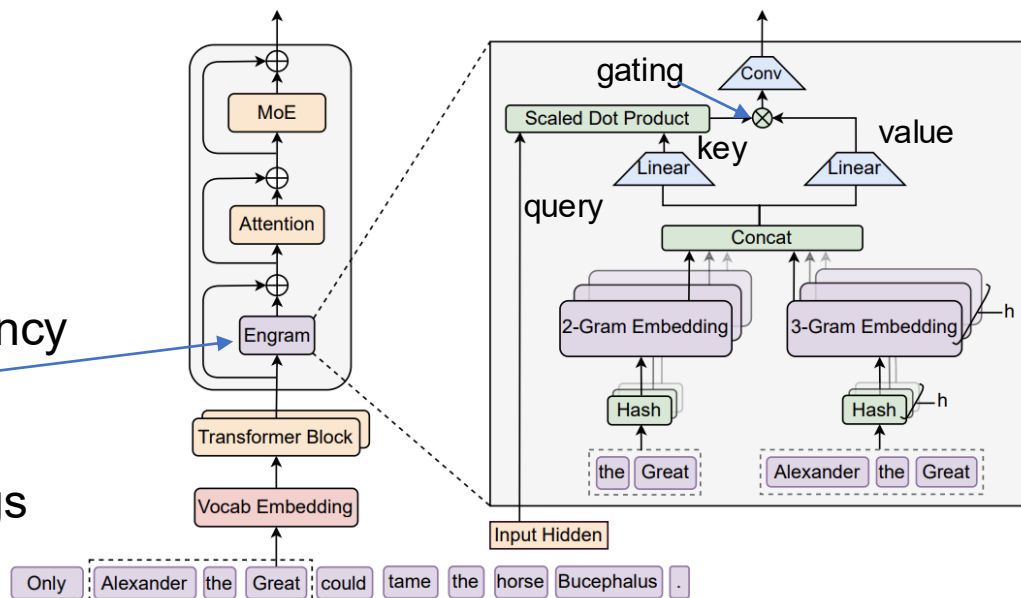
3. The retrieved neighbors are passed through a **Transformer Encoder**. This encoder turns the raw text of the neighbors into mathematical representations (embeddings) that the main model can understand.

4. **Chunked Cross-Attention (CCA)** combines its own current thoughts with the information it found in the database.



Conditional Memory via Scalable Lookup: A New Axis of Sparsity for LLMs

- Transformers currently lack a native primitive for knowledge lookup, forcing them to "inefficiently simulate retrieval through computation."
 - Engram addresses this inefficiency by introducing a scalable and **lookup** mechanism based on modernized N-gram embeddings which allows for significant parameter scaling with minimal inference overhead.



System Implementation of Engram

- During training, model parallelism is utilized by sharding large-scale embedding tables across multiple GPUs. This design allows memory capacity to scale linearly with the number of accelerators.
- During inference, Engram tables can be off-loaded to host memory
 - The key idea of "Offloaded Engram" is to store the massive static memory table in CPU Host Memory (RAM) instead of the limited and expensive GPU Memory (HBM), while maintaining high speed through prefetching.

The Engram layer acts as a static knowledge retrieval system. Instead of "thinking" about the input, it simply "looks up" the answer based on the text pattern.

The MoE layer acts as the dynamic reasoning engine. It processes information to understand logic and context.

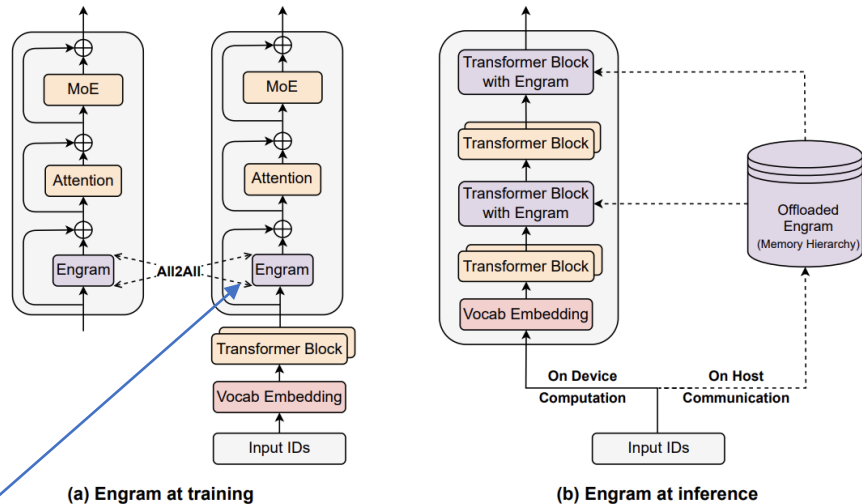


Figure 2 | **System implementation of Engram.** (a) Training Phase: The massive embedding tables are sharded across available GPUs. An All-to-All communication primitive is employed to retrieve active embedding rows across devices. (b) Inference Phase: Engram tables are off-loaded to host memory. By exploiting the deterministic retrieval logic, the host asynchronously prefetches and transfers embeddings, overlapping communication with the on-device computation of preceding Transformer blocks.

Summary

